МИНОБРНАУКИ РОССИИ

федеральное государственное бюджетное образовательное учреждение
высшего образования
«Санкт-Петербургский государственный технологический институт
(технический университет)»

УГСН	09.00.00	Информатика и вычислительная техника
Уровень образования		Высшее образование – бакалавриат
Форма обучения		Очная
Факультет		Информационных технологий и управления
Кафедра		Систем автоматизированного проектирования и управления
Учебная дисциплина		Информационные технологии и программирование

Курс I Группа 4503

Отчёт по контрольной работе № 1

Вариант № 23

Исполнитель:
обучающийся
группы 4503 _____ Малый Илья Романович
(дата, подпись)

Проверил: _____ Корниенко Иван Григорьевич
(дата, подпись) Федин Алексей Константинович
Плеханов Александр Андреевич

Санкт-Петербург
2026

СОДЕРЖАНИЕ

1 Постановка задачи 3
2 Исходные данные 3
3 Особые ситуации 4
4 Математические методы и алгоритмы решения задач 4
5 Форматы представления данных 5
6 Структура программы 6
7 Описание хода выполнения контрольной работы 7
8 Результаты работы программы 10
9 Исходный текст программы 12

1 Постановка задачи

Разработать класс для указанной предметной области. Доступ к данным реализовать с помощью методов Set, Get, Show. Предусмотреть необходимые проверки исходных данных, а также возможность получения указанных в задании списков на основе текущих исходных данных в произвольном порядке и любое число раз.

Класс – автобус:

1) номер автобуса,
2) номер маршрута,
3) марка,
4) год начала эксплуатации,
5) пробег,
6) фамилия и инициалы водителя.

Создать массив объектов. Реализовать возможность получения:

1) списка автобусов для заданного номера маршрута,
2) списка автобусов, которые эксплуатируются больше 10 лет,
3) списка автобусов, пробег у которых больше 10 000 км.

2 Исходные данные

В качестве исходных данных создадим 10 автобусов: 4 различных маршрута – натуральное число, год эксплуатации выберем случайно в диапазоне от 2010 до 2026, аналогично выберем пробег в диапазоне от 5000 км до 15000 км, номер автобуса должен быть уникальным в формате *A111AA*: где *A* – одна из 12 букв кириллицы, которые совпадают с латиницей, *1* – любая цифра; фамилии и инициалы водителей так же выберем случайно, будет представлять из себя одну строку в формате *Фамилия А. Б.* : где *Фамилия* – любой набор символов, где первая буква – заглавная, *А. Б.* – любые две буквы с последующей точкой и пробелом перед вторым инициалом, маркой будет любой набор символов.

Выбранные автобусы представлены в таблице 1.

Таблица 1 – Список автобусов

№ п/п	№ автобуса	№ маршрута	Марка	Год начала	Пробег (км)	Фамилия, инициалы
1	В522ОУ	68	Volgabus	2016	10092	Яковлев И. А.
2	Е122ОМ	68	ПАЗ	2012	14371	Матвеева В. А.
3	С643МН	68	КАМАЗ	2010	8681	Алексеев М. М.
4	К263ВТ	68	Mercedes	2015	13930	Гущина П. И.
5	Т770КС	242	НЕФАЗ	2013	6250	Царев К. Л.
6	У585ХТ	242	Scania	2014	8566	Белов М. М.
7	М875АХ	242	ЛиАЗ	2019	6756	Беспалов М. С.
8	О852РН	297	МАЗ	2017	12644	Семенов Р. А.
9	У445ЕО	297	Volgabus	2022	7125	Савин М. А.
10	О441ХВ	130	ПАЗ	2026	14334	Софронов С. Д.

3 Особые ситуации

Поскольку возможность добавления автобусов не является требованием конкретной задачи, но является общим требованием контрольных работ, необходимо осуществлять проверку данных перед непосредственным добавлением:

- номер должен соответствовать формату,
- номер маршрута должен быть натуральным числом,
- год начала эксплуатации должен быть натуральным числом,
- пробег должен быть вещественным числом,
- фамилия и инициалы должны соответствовать формату.

Взаимодействие с пользователем будет происходить в виде меню с возможностью выбирать пункты. В этом случае нужно будет проверить ввод на то, соответствует ли он какому-либо пункту, в ином случае сообщить пользователю о некорректном вводе. Также для получения списка автобусов заданного маршрута необходимо получить этот маршрут, а значит и проверить является ли пользовательский ввод натуральным числом.

4 Математические методы и алгоритмы решения задач

Для получения списка автобусов заданного маршрута нужно будет пройти по всему списку автобусов и выбрать те, номер которых совпадает с выбранным. Это будет опцией 1 в меню выбора.

Так же и для списка автобусов, срок эксплуатации которых больше десяти лет – пройдем по всему списку и составим список, удовлетворяющих условию. Для этого нужно будет получить текущий год. Будем считать, что если сейчас 2001 год, то автобус с 2000 годом начала эксплуатации уже работает один год, несмотря на то, что с момента начала его работы могло пройти не больше пары месяцев. Это будет опцией 2 в меню выбора.

Точно так же пройдем по всему списку и выделим автобусы с пробегом больше 10000 километров. Это будет опцией 3 в меню выбора.

Блок-схема алгоритмов работы программы представлена на рисунке 1.

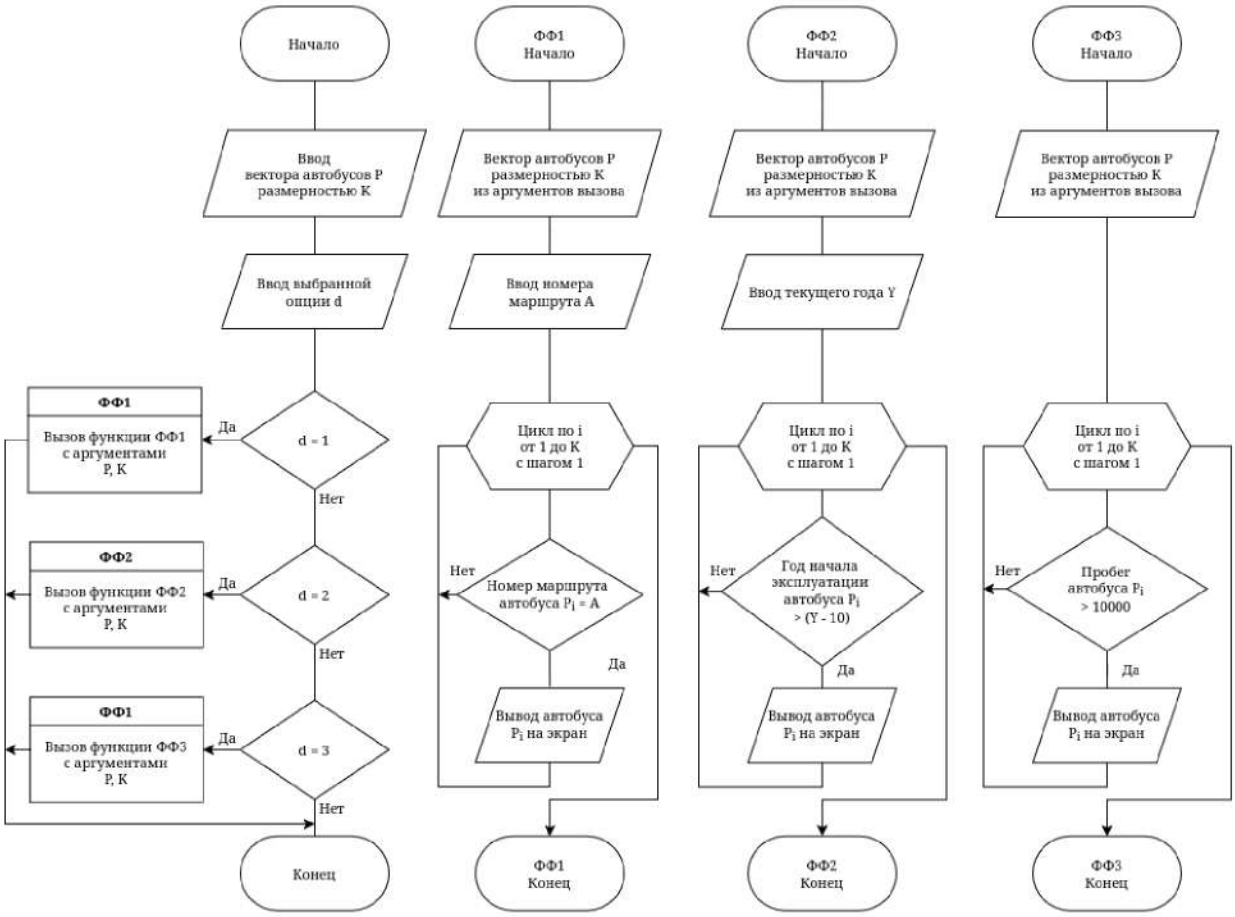

Рисунок 1 – Блок-схема алгоритмов программы

5 Форматы представления данных

Типы данных класса Bus представлены в таблице 2.

Таблица 2 – Типы данных свойств класса Bus

Имя	Тип	Описание
number	string	Номер автобуса
route_number	int	Номер маршрута
brand	string	Марка
exploitation_start_year	int	Год начала эксплуатации
kilometrage	double	Пробег
driver	string	Водитель

6 Структура программы

Программа была разделена на модули:

1) main.cpp – точка входа и установка необходимых параметров,

2) utils.cpp/utils.h – общие функции для заданий,

3) console_interface.cpp/console_interface.h – консольный интерфейс программы (консольные меню),

4) task_1.cpp/task_1.h – алгоритмы и класс автобуса для первой контрольной работы.

Функции в файле utils.cpp представлены в таблице 3.

Таблица 3 – Функции, опрелеленные в utils.cpp

Название функции	Тип аргументов	Возвращаемый тип	Описание
read_int	–	int	Берет число из пользовательского ввода и возвращает его в случае отсутствия ошибок преобразований
read_natural	–	int	Берет натуральное число из пользовательского ввода
read_double	–	double	Берет вещественное число из пользовательского ввода

Функции в файле test_1.cpp представлены в таблице 4.

Таблица 4 – Функции, опрелеленные в test_1.cpp

Название функции	Тип аргументов	Возвращаемый тип	Описание
get_buses_by_route	list<Bus>, int	list<Bus>	Возвращает список автобусов с номером переданном в качестве аргумента из полученного списка
get_buses_over_10_years	list<Bus>	list<Bus>	Возвращает список автобусов со сроком эксплуатации больше 10 лет из полученного списка
get_buses_over_10k_km	list<Bus>	list<Bus>	Возвращает список автобусов с пробегом больше 10 000 километров из полученного списка
print_buses	list<Bus>	–	Распечатывает полученный на вход список автобусов
add_bus	list<Bus>	–	Добавляет автобус в полученный список
remove_buses	list<Bus>	–	Очищает список автобусов

7 Описание хода выполнения контрольной работы

В самом начале был создан проект в среде Visual Studio, включающий в себя инструмент CMake. Был выбран именно такой вариант, чтобы обеспечить совместимость с другими операционными системами. Затем был проинициализирован репозиторий git для обеспечения удобства работы на нескольких устройствах (виртуальных машинах).

Первым в коде была разработана функция read_int(), которая читает целое число из пользовательского ввода и при необходимости указывает на некорректность ввода.

Далее было разработано меню выбора контрольных работ, так же с проверкой на корректность и наличие введенного пункта.

На этом этапе было замечено, что символы кириллицы отображаются некорректно если проект компилировать в операционной системе Windows в среде разработки Visual Studio. В операционной системе Linux такой проблемы не было. Решением стало добавление библиотеки windows.h в файл main.cpp (листинг 1) и смена кодировки консоли на CP_UTF8, если объявлена константа _MSC_VER (позже будет изменено) или при условии, что проект компилируется в среде разработки Visual Studio в тот же файл, но внутри функции main (листинг 2).

Листинг 1 – Добавление библиотеки windows.h

```
#ifdef _MSC_VER
#include <windows.h>
#endif
```

Листинг 2 – Смена кодировки консоли на CP_UTF8

```
#ifdef _MSC_VER
SetConsoleOutputCP(CP_UTF8);
#endif
```

Следом была разработана предполагаемая структура программы – подробнее описано в разделе 6.

После этого был разработан класс Bus, описывающий автобус включая методы set для изменения значений свойств (полей) объекта, методы get для получения значений свойств и show для текстового описания объекта.

Затем было разработано меню выбора опций:

1) получить список автобусов с определенным маршрутом,
2) получить список автобусов со сроком эксплуатации больше 10 лет,
3) получить список автобусов с пробегом больше 10к км,
4) вывести полученный список,
5) вывести список всех автобусов,
6) добавить автобус,
7) очистить список автобусов,
8) выйти из задания.

Следом за этим – реализация алгоритмов соответствующих опций. Контейнером хранения списков автобусов был выбран std::list поскольку он в своей реализации представляет из себя связный список, что позволяет быстро добавлять элементы, но из-за этого получение элемента по индексу занимает больше времени, чем в обычном массиве или векторе std::vector, однако все алгоритмы завязаны на переборе массива и получении подходящих элементов с дальнейшим их добавлением в результирующий массив, а с этим связный список справляется хорошо.

Алгоритмы были реализованы в виде функций get_buses_by_route / get_buses_over_10_years / get_buses_over_10k_km соответственно, которые на вход принимают весь массив автобусов, а в качестве результата возвращают составленный список, который сохраняется в переменную got_buses и следом их можно распечатать с помощью опции 4. Аналогично работает вывод всех автобусов.

Отдельное внимание уделено алгоритму добавления автобусов, который, не предусмотрен по заданию, но является общим требованием для всех контрольных работ:

• Получить номер автобуса (в формате A111AA), просим пользователя ввести его и обрабатываем с помощью технологии регулярных выражений (regular expressions, RegEx), сверяем на соответствие шаблону `^[АВЕКМНОРСТУХ]\d{3}[АВЕКМНОРСТУХ]{2}$` , где ^ и $ – начало и конец строки соответственно, `[АВЕКМНОРСТУХ]` – один символ из этого списка допустимых символов для номера (буквы латиницы, которые есть в кириллице), `\d{3}` – три десятичных цифры, `[АВЕКМНОРСТУХ]{2}` – два символа из этого списка.

• Получить номер маршрута, для этого была реализована функция get_natural, которая вызывает функцию get_int и проверяет было ли получено положительное число.

• Получить марку автобуса, может быть любой непустой строкой.

• Получить год начала эксплуатации, осуществляется с помощью функции get_natural.

• Получить пробег, может быть вещественным неотрицательным числом, для его получения была разработана функция get_double, а также осуществляется проверка на неотрицательность.

• Получить фамилию и инициалы водителя, строка из пользовательского ввода с проверкой на соответствие regex шаблону: `^[A-Z][a-z]* [A-Z]\. [A-Z]\.$` , где `[A-Z]` – одна заглавная латинская буква, `[a-z]*` – любое (в том числе нулевое) количество строчных латинских букв, `\.` – точка (является спец символом в regex и требует экранирования)

На этом этапе были выявлены недостатки ввода символов через консоль в ОС Windows, в программу консоль отправляет символы в однобайтовой кодировке, а символы кириллицы в кодировке программы представляют из себя последовательности из двух байтов, в результате введенные символы заменялись на `'\0'` в связи с этим было принято решение убрать возможность ввода на кириллице.

Наконец, была добавлена возможность очистки списка автобусов.

Были получены комментарии к работе в состоянии на тот момент от преподавателя, выявлены недочеты в работе. Были внесены следующие изменения: вместо константы _MSC_VER выбрана _WIN32, которая задана при компиляции в системе Windows, независимо от среды разработки; все

названия, содержащие test были переименованы в аналогичные со словом task; функции, отвечающие за консольное меню в каком бы то ни было виде были отделены от алгоритмов задания, в частности функция выбора контрольных работ и опций первой контрольных работ были вынесены в файл console_interface.cpp, в файле task_1.cpp/h была убрана зависимость от консоли, метод show теперь возвращает строку, а не выводит её в консоль, проверки данных на корректность теперь осуществляется в сеттерах, а не в функции добавления автобуса в список.

8 Результаты работы программы

Тестирование программы было произведено на исходных данных из таблицы 1. На рисунках 2, 3, 4, 5 продемонстрировано вывод полного списка автобусов, списка автобусов 297 маршрута, списка автобусов со сроком эксплуатации больше 10 лет, списка автобусов с пробегом больше 10 000 км соответственно.

```
1) Получить список автобусов для заданного номера маршрута
2) Получить список автобусов, которые эксплуатируются больше 10 лет
3) Получить список автобусов, пробег у которых больше 10 000 км
4) Вывести полученный список
5) Вывести список всех автобусов
6) Добавить автобус
7) Удалить все автобусы из списка
0) Вернуться в меню выбора контрольных работ

Выберите соответствующий пункт меню: 5
Номер автобуса: B522OY Номер маршрута: 68 Марка: "Volgabus" Год начала эксплуатации: 2016 Пробег: 10092 Водитель: "Yakovlev I. A."
Номер автобуса: E122OM Номер маршрута: 68 Марка: "ПАЗ" Год начала эксплуатации: 2012 Пробег: 14371 Водитель: "Matveeva V. A."
Номер автобуса: C643MH Номер маршрута: 68 Марка: "КАМАЗ" Год начала эксплуатации: 2010 Пробег: 8681 Водитель: "Alexeev M. M."
Номер автобуса: K263BT Номер маршрута: 68 Марка: "Mercedes" Год начала эксплуатации: 2015 Пробег: 13930 Водитель: "Gushina P. I."
Номер автобуса: T770KC Номер маршрута: 242 Марка: "НЕФАЗ" Год начала эксплуатации: 2013 Пробег: 6250 Водитель: "Tsarev K. L."
Номер автобуса: Y585XT Номер маршрута: 242 Марка: "Scania" Год начала эксплуатации: 2014 Пробег: 8566 Водитель: "Belov M. M."
Номер автобуса: M875AX Номер маршрута: 242 Марка: "ЛиАЗ" Год начала эксплуатации: 2019 Пробег: 6756 Водитель: "Bespalov M. C."
Номер автобуса: O852PH Номер маршрута: 297 Марка: "МАЗ" Год начала эксплуатации: 2017 Пробег: 12644 Водитель: "Semenov P. A."
Номер автобуса: Y445EO Номер маршрута: 297 Марка: "Volgabus" Год начала эксплуатации: 2022 Пробег: 7125 Водитель: "Savin M. A."
Номер автобуса: O441XB Номер маршрута: 130 Марка: "ПАЗ" Год начала эксплуатации: 2026 Пробег: 14334 Водитель: "Sofronov S. D."
```

Рисунок 2 – Результат вывода всего списка

```
Выберите соответствующий пункт меню: 1
Введите номер маршрута (натуральное число): 297
Успешно! Вывести полученный список можно выбрав пункт 4

1) Получить список автобусов для заданного номера маршрута
2) Получить список автобусов, которые эксплуатируются больше 10 лет
3) Получить список автобусов, пробег у которых больше 10 000 км
4) Вывести полученный список
5) Вывести список всех автобусов
6) Добавить автобус
7) Удалить все автобусы из списка
0) Вернуться в меню выбора контрольных работ

Выберите соответствующий пункт меню: 4
Номер автобуса: O852PH Номер маршрута: 297 Марка: "МАЗ" Год начала эксплуатации: 2017 Пробег: 12644 Водитель: "Semenov P. A."
Номер автобуса: Y445EO Номер маршрута: 297 Марка: "Volgabus" Год начала эксплуатации: 2022 Пробег: 7125 Водитель: "Savin M. A."
```

Рисунок 3 – Результат вывода автобусов 297 маршрута

```
Выберите соответствующий пункт меню: 2
Успешно! Вывести полученный список можно выбрав пункт 4

1) Получить список автобусов для заданного номера маршрута
2) Получить список автобусов, которые эксплуатируются больше 10 лет
3) Получить список автобусов, пробег у которых больше 10 000 км
4) Вывести полученный список
5) Вывести список всех автобусов
6) Добавить автобус
7) Удалить все автобусы из списка
0) Вернуться в меню выбора контрольных работ

Выберите соответствующий пункт меню: 4
Номер автобуса: E122OM Номер маршрута: 68 Марка: "ПАЗ" Год начала эксплуатации: 2012 Пробег: 14371 Водитель: "Matveeva V. A."
Номер автобуса: C643MH Номер маршрута: 68 Марка: "КАМАЗ" Год начала эксплуатации: 2010 Пробег: 8681 Водитель: "Alexeev M. M."
Номер автобуса: K263BT Номер маршрута: 68 Марка: "Mercedes" Год начала эксплуатации: 2015 Пробег: 13930 Водитель: "Gushina P. I."
Номер автобуса: T770KC Номер маршрута: 242 Марка: "НЕФАЗ" Год начала эксплуатации: 2013 Пробег: 6250 Водитель: "Tsarev K. L."
Номер автобуса: Y585XT Номер маршрута: 242 Марка: "Scania" Год начала эксплуатации: 2014 Пробег: 8566 Водитель: "Belov M. M."
```

Рисунок 4 – Результат вывода автобусов больше 10 лет эксплуатации

```
Выберите соответствующий пункт меню: 3
Успешно! Вывести полученный список можно выбрав пункт 4

1) Получить список автобусов для заданного номера маршрута
2) Получить список автобусов, которые эксплуатируются больше 10 лет
3) Получить список автобусов, пробег у которых больше 10 000 км
4) Вывести полученный список
5) Вывести список всех автобусов
6) Добавить автобус
7) Удалить все автобусы из списка
0) Вернуться в меню выбора контрольных работ

Выберите соответствующий пункт меню: 4
Номер автобуса: B522OY Номер маршрута: 68 Марка: "Volgabus" Год начала эксплуатации: 2016 Пробег: 10092 Водитель: "Yakovlev I. A."
Номер автобуса: E122OM Номер маршрута: 68 Марка: "ПАЗ" Год начала эксплуатации: 2012 Пробег: 14371 Водитель: "Matveeva V. A."
Номер автобуса: K263BT Номер маршрута: 68 Марка: "Mercedes" Год начала эксплуатации: 2015 Пробег: 13930 Водитель: "Gushina P. I."
Номер автобуса: O852PH Номер маршрута: 297 Марка: "МАЗ" Год начала эксплуатации: 2017 Пробег: 12644 Водитель: "Semenov P. A."
Номер автобуса: O441XB Номер маршрута: 130 Марка: "ПАЗ" Год начала эксплуатации: 2026 Пробег: 14334 Водитель: "Sofronov S. D."
```

Рисунок 5 – Результат вывода списка автобусов с пробегом больше 10 000 км

9 Исходный текст программы

```
// main.cpp
// Общий файл контрольных работ
// Студент группы 4503, Илья М, 2026

#include "console_interface.h"
#ifdef _WIN32
#include <Windows.h>
#endif


int main() {
#ifdef _WIN32
    system("chcp 65001");
    system("cls");
#endif

    main_menu();

    return 0;
}

// utils.cpp
// Общий файл контрольных работ
// Студент группы 4503, Илья М, 2026

#include "utils.h"
#include "task_1.h"

#include <iostream>
#include <string>


int read_int() {
    std::string input;
    bool success = false;
    int result = 0;

    do {
        try {
            std::getline(std::cin, input);
            result = std::stoi(input);
            success = true;
        } catch (std::invalid_argument &) {
            std::cout << "Некорректный ввод. Попробуйте ещё раз: ";
        }
    } while (!success);

    return result;
}

double read_double() {
    std::string input;
    bool success = false;
    double result = 0;

    do {
        try {
            std::getline(std::cin, input);
            result = std::stod(input);
            success = true;
        } catch (std::invalid_argument &) {
            std::cout << "Некорректный ввод. Попробуйте ещё раз: ";
```

```
        }
    } while (!success);

    return result;
}

int read_natural() {
    int result = 0;
    bool valid = false;
    do {
        result = read_int();
        valid = (result > 0);
        if (!valid) std::cout << "Полученное число не натуральное. Попробуйте
ещё раз: ";
    } while (!valid);
    return result;
}

// utils.h
// Общий файл контрольных работ
// Студент группы 4503, Илья М, 2026

#ifndef UTILS_H
#define UTILS_H

int read_int();
double read_double();
int read_natural();

#endif //UTILS_H

// console_interface.cpp
// Консольный интерфейс программы
// Студент группы 4503, Илья М, 2026

#include "console_interface.h"

#include "task_1.h"
#include "utils.h"

#include <iostream>
#include <list>

void main_menu() {
    bool continue_cycle = true;
    do {
        std::cout << std::endl;
        std::cout << "1) Выбрать контрольную работу 1" << std::endl;
        // std::cout << "2) Выбрать контрольную работу 2" << std::endl;
        // std::cout << "3) Выбрать контрольную работу 3" << std::endl;
        std::cout << "0) Выход" << std::endl;
        std::cout << std::endl;
        std::cout << "Выберите соответствующий пункт меню: ";

        bool input_again = false;
        do {
            input_again = false;
            switch (int decision = read_int()) {
            case OPTION_TEST_1:
                task_1_menu();
                break;

            case OPTION_EXIT:
                continue_cycle = false;
```

```
                break;

            default:
                std::cout << "Нет такого пункта меню. Попробуйте ещё раз: ";
                input_again = true;
            }
        } while (input_again);
    } while (continue_cycle);
}

void task_1_menu() {
    std::list<Bus> buses;
    std::list<Bus> got_buses;


    Bus sample_bus;
    sample_bus.set_number(SAMPLE_BUS_NUMBER);
    sample_bus.set_route_number(SAMPLE_BUS_ROUTE);
    sample_bus.set_brand(SAMPLE_BUS_BRAND);
    sample_bus.set_exploitation_start_year(SAMPLE_BUS_YEAR);
    sample_bus.set_kilometrage(SAMPLE_BUS_KM);
    sample_bus.set_driver(SAMPLE_BUS_DRIVER);
    buses.push_back(sample_bus);
    bool continue_cycle = true;
    do {
        std::cout << std::endl;
        std::cout << "1) Получить список автобусов для заданного номера
маршрута" << std::endl;
        std::cout << "2) Получить список автобусов, которые эксплуатируются
больше 10 лет" << std::endl;
        std::cout << "3) Получить список автобусов, пробег у которых больше 10
000 км" << std::endl;
        std::cout << "4) Вывести полученный список" << std::endl;
        std::cout << "5) Вывести список всех автобусов" << std::endl;
        std::cout << "6) Добавить автобус" << std::endl;
        std::cout << "7) Удалить все автобусы из списка" << std::endl;
        std::cout << "0) Вернуться в меню выбора контрольных работ" <<
std::endl;
        std::cout << std::endl;
        std::cout << "Выберите соответствующий пункт меню: ";

        bool input_again = false;
        do {
            int selected_route = 0;
            input_again = false;
            int decision = read_int();
            switch (decision) {
            case OPTION_BUSES_BY_ROUTE:
                std::cout << "Введите номер маршрута (натуральное число): ";
                selected_route = read_natural();
                got_buses = get_buses_by_route(buses, selected_route);
                std::cout << "Успешно! Вывести полученный список можно выбрав
пункт 4" << std::endl;
                break;

            case OPTION_BUSES_OVER_10_YEARS:
                got_buses = get_buses_over_10_years(buses);
                std::cout << "Успешно! Вывести полученный список можно выбрав
пункт 4" << std::endl;
                break;

            case OPTION_BUSES_OVER_10K_KM:
                got_buses = get_buses_over_10k_km(buses);
```

```
                std::cout << "Успешно! Вывести полученный список можно выбрав
пункт 4" << std::endl;
                break;

            case OPTION_PRINT_LIST:
                print_buses(got_buses);
                break;

            case OPTION_PRINT_ALL:
                print_buses(buses);
                break;

            case OPTION_ADD_BUS:
                add_bus(buses);
                break;

            case OPTION_EMPTY_BUS_LIST:
                remove_buses(buses);
                break;

            case OPTION_EXIT:
                continue_cycle = false;
                break;

            default:
                std::cout << "Нет такого пункта меню. Попробуйте ещё раз: ";
                input_again = true;
            }
        } while (input_again);
    } while (continue_cycle);
}

void print_buses(const std::list<Bus>& buses) {
    if (buses.empty()) {
        std::cout << "Список пуст" << std::endl;
    } else {
        for (const Bus& i : buses) {std::cout << i.show() << std::endl;}
    }
}
void add_bus(std::list<Bus>& buses) {

    Bus bus_to_add;

    bool success = false;

    std::cout << "Введите номер автобуса в формате \"A111AA\" (латинскими
символами, совпадающими с кириллицей): ";
    do {
        std::string number;
        std::getline(std::cin, number);
        success = bus_to_add.set_number(number);
        if (!success) std::cout << "Введенное значение \"" << number << "\" не
соответствует требованиям. Попробуйте ещё раз: ";
    } while (!success);

    std::cout << "Введите номер маршрута (натуральное число): ";
    do {
        int route = read_int();
        success = bus_to_add.set_route_number(route);
        if (!success) std::cout << "Введенное значение \"" << route << "\" не
соответствует требованиям. Попробуйте ещё раз: ";
    } while (!success);

    std::cout << "Введите марку автобуса: ";
```

```
    do {
        std::string brand;
        std::getline(std::cin, brand);
        success = bus_to_add.set_brand(brand);
        if (!success) std::cout << "Введенное значение \"" << brand << "\" не
соответствует требованиям. Попробуйте ещё раз: ";
    } while (!success);

    std::cout << "Введите год начала эксплуатации автобуса (положительное целое
число): ";
    do {
        int year = read_int();
        success = bus_to_add.set_exploitation_start_year(year);
        if (!success) std::cout << "Введенное значение \"" << year << "\" не
соответствует требованиям. Попробуйте ещё раз: ";
    } while (!success);

    std::cout << "Введите пробег автобуса (вещественное положительное число):
";
    do {
        double kilometrage = read_double();
        success = bus_to_add.set_kilometrage(kilometrage);
        if (!success) std::cout << "Введенное значение \"" << kilometrage <<
"\" не соответствует требованиям. Попробуйте ещё раз: ";
    } while (!success);

    std::cout << "Введите фамилию и инициалы водителя автобуса в формате Фамилия
А. Б. (латинскими символами): ";
    do {
        std::string driver;
        std::getline(std::cin, driver);
        success = bus_to_add.set_driver(driver);
        if (!success) std::cout << "Введенное значение \"" << driver << "\" не
соответствует формату. Попробуйте ещё раз: ";
    } while (!success);


    buses.push_back(bus_to_add);
}
void remove_buses(std::list<Bus>& buses) {
    buses.clear();
    print_buses(buses);
}

// console_interface.h
// Консольный интерфейс программы
// Студент группы 4503, Илья М, 2026

#ifndef ITAP_TESTS_MAIN_MENU_H
#define ITAP_TESTS_MAIN_MENU_H
#include "task_1.h"

enum main_menu_options {
    OPTION_EXIT = 0,
    OPTION_TEST_1 = 1,
    OPTION_TEST_2 = 2,
    OPTION_TEST_3 = 3
};

enum test_1_menu_options {
    OPTION_BUSES_BY_ROUTE = 1,
    OPTION_BUSES_OVER_10_YEARS = 2,
    OPTION_BUSES_OVER_10K_KM = 3,
    OPTION_PRINT_LIST = 4,
```

```
    OPTION_PRINT_ALL = 5,
    OPTION_ADD_BUS = 6,
    OPTION_EMPTY_BUS_LIST = 7,
};
void main_menu();

void task_1_menu();
void print_buses(const std::list<Bus> & buses);
void add_bus(std::list<Bus>& buses);
void remove_buses(std::list<Bus>& buses);


#endif // ITAP_TESTS_MAIN_MENU_H

// task_1.cpp
// Контрольная работа №1
// Студент группы 4503, Илья М, 2026

#include "task_1.h"
#include <list>
#include <regex>
#include <ctime>

std::string Bus::show() const {
    std::string result_string =
        "Номер автобуса: " + get_number() +
        " Номер маршрута: " + std::to_string(get_route_number()) +
        " Марка: \"" + get_brand() +
        "\"        Год        начала        эксплуатации:        "        +
std::to_string(exploitation_start_year) +
        " Пробег: " + std::to_string(kilometrage) +
        " Водитель: \"" + get_driver();
    return result_string;
}

bool Bus::set_number(std::string p_number) {
    if (std::regex_match(p_number, number_pattern)) {
        number = std::move(p_number);
        return true;
    }
    return false;
}
bool Bus::set_route_number(int p_route) {
    if (p_route > 0) {
        route_number = p_route;
        return true;
    };
    return false;
}
bool Bus::set_brand(std::string p_brand) {
    if (!p_brand.empty()) {
        brand = std::move(p_brand);
        return true;
    }
    return false;
}
bool Bus::set_exploitation_start_year(int p_year) {
    if (p_year >= 0) {
        exploitation_start_year = p_year;
        return true;
    };
    return false;
}
bool Bus::set_kilometrage(double p_km) {
```

```
    if (p_km >= 0) {
        kilometrage = p_km;
        return true;
    };
    return false;
}
bool Bus::set_driver(std::string p_driver) {
    if (std::regex_match(p_driver, driver_pattern)) {
        driver = std::move(p_driver);
        return true;
    }
    return false;
}

std::list<Bus> get_buses_by_route(const std::list<Bus>& buses, int route) {
    std::list<Bus> buses_by_route;
    for (const Bus& i : buses) {
        if (i.get_route_number() == route) buses_by_route.push_back(i);
    }
    return buses_by_route;
}
std::list<Bus> get_buses_over_10_years(const std::list<Bus>& buses) {
    std::list<Bus> buses_over_10_years;
    for (const Bus& i : buses) {
        if (i.get_exploitation_start_year() < (CURRENT_YEAR - YEARS_THRESHOLD))
buses_over_10_years.push_back(i);
    }
    return buses_over_10_years;
}
std::list<Bus> get_buses_over_10k_km(const std::list<Bus>& buses) {
    std::list<Bus> buses_over_10k_km;
    for (const Bus& i : buses) {
        if         (i.get_kilometrage()         >         KILOMETERS_THRESHOLD)
buses_over_10k_km.push_back(i);
    }
    return buses_over_10k_km;
}

// task_1.h
// Контрольная работа №1
// Студент группы 4503, Илья М, 2026

#ifndef ITAP_TESTS_TASK_1_H
#define ITAP_TESTS_TASK_1_H
#include <regex>
#include <string>
#include <list>

#define YEARS_THRESHOLD 10
#define KILOMETERS_THRESHOLD 10000

#define SECONDS_PER_YEAR (60*60*24*365)
#define EPOCH_START 1970

#define SAMPLE_BUS_NUMBER "B522OY"
#define SAMPLE_BUS_ROUTE 68
#define SAMPLE_BUS_BRAND "Volgabus"
#define SAMPLE_BUS_YEAR 2025
#define SAMPLE_BUS_KM 4321
#define SAMPLE_BUS_DRIVER "Yakovlev I. A."


class Bus {
    std::string number;
```

```
    int route_number;
    std::string brand;
    int exploitation_start_year;
    double kilometrage;
    std::string driver;
    const              std::regex              number_pattern              =
std::regex("^[ABEKMHOPCTYX]\\d{3}[ABEKMHOPCTYX]{2}$");
    const std::regex driver_pattern = std::regex("^[A-Z][a-z]* [A-Z]\\. [A-
Z]\\.$");

    public:
    Bus() :
    route_number(0), exploitation_start_year(0), kilometrage(0)
    {}

    std::string get_number() const {return number;}
    int get_route_number() const {return route_number;}
    std::string get_brand() const {return brand;}
    int get_exploitation_start_year() const {return exploitation_start_year;}
    double get_kilometrage() const {return kilometrage;}
    std::string get_driver() const {return driver;}

    bool set_number(std::string p_number);
    bool set_route_number(int p_route);
    bool set_brand(std::string p_brand);
    bool set_exploitation_start_year(int p_year);
    bool set_kilometrage(double p_km);
    bool set_driver(std::string p_driver);

    std::string show() const;

};

std::list<Bus> get_buses_by_route(const std::list<Bus>& buses, int route);
std::list<Bus> get_buses_over_10_years(const std::list<Bus>& buses);
std::list<Bus> get_buses_over_10k_km(const std::list<Bus>& buses);

const int CURRENT_YEAR = static_cast<int> (EPOCH_START+ std::time(nullptr) /
SECONDS_PER_YEAR);

#endif // ITAP_TESTS_TASK_1_H
```